MonkeyBot: A Planning System for Adhesion-Free Multi-Limb Robotic Climbing

Anonymous submission

Abstract

We demonstrate *MonkeyBot*, an end-to-end planning system for adhesion-free multi-limbed robotic climbing. Unlike prior climbing robots that assume stable anchoring at arbitrary surface locations, MonkeyBot targets sparse, directionally constrained contact points and terrain gaps that require dynamic maneuvers such as ballistic jumps. The system reduces the continuous climbing task to a discrete AI planning problem via three tightly integrated components: a precomputation module that validates dynamic maneuvers and encodes them as symbolic *Transition Links* (TLs); a classical planner operating over the resulting FDR task; and a physics-based execution engine that translates high-level plans into robot motion. To keep planning tractable, we introduce three satisfiability-preserving pruning strategies that reduce the TL action set by up to 95%. Experiments across 12 benchmark instances show that pruning enables the planner to solve problems it could not otherwise handle within a 600-second timeout, with the most effective strategy alone yielding a near-threefold speedup. A video demonstration is available at: https://www.youtube.com/watch?v=exAnQH9lMtI

Introduction

Steep and irregular terrain remains inaccessible to wheeled rovers. Existing climbing robots such as the LEMUR series (Parness et al. 2017), LORIS (Nadan, Backus, and Johnson 2024), and SCALER (Tanaka et al. 2022) rely on low-level execution pipelines that assume broadly grippable or pre-mapped surfaces. When this assumption fails, even a single misgrip can be mission-ending.

We introduce *MonkeyBot*, a planning architecture that unifies deterministic planning, dynamic feasibility checking, and low-level execution. The system makes three contributions. First, a planning model for adhesion-free climbing: the robot is modeled as a body with n limbs over a set of discrete gripping points, yielding a Finite Domain Representation (FDR) task whose actions include limb release, reattachment, and body shifts. This allows off-the-shelf classical planners to generate high-level climbing strategies with no domain-specific search modifications. Second, *Transition Links* (TLs): symbolic actions encoding dynamically feasible maneuvers such as ballistic jumps. Their feasibility is governed by nonlinear constraints that cannot be encoded directly in a symbolic planner, so each TL is validated offline and injected into the planning instance as an ordinary action. Third, pruning strategies that reduce the TL action set by up to 95% while preserving solvability, making planning tractable on larger instances. The full system is demonstrated in a 2D physics simulator (PyMunk), where a controller maps planner output to limb-level execution.

System Description

Robot Model and Planning Domain. The virtual robot consists of a rigid body center connected to n extensible limbs. Each limb can independently extend and retract, attaching to or releasing from discrete gripping points on a 2D climbing surface. The climbing environment is represented as a set of gripping points $V \subset \mathbb{R}^2$, each a location on the wall where a limb tip can latch. A limb can reach any gripping point within Euclidean distance E of the body center. The robot must keep at least k limbs attached at all times to remain stable.

This physical setup maps directly to a classical planning task in the Finite Domain Representation (FDR) (Helmert 2006), formulated in the Unified Planning Framework (Micheli et al. 2025). Each limb has a state variable ranging over $V \cup \{\emptyset\}$, recording which gripping point it currently holds (or that it is detached). The action set consists of $\text{ATTACH}_i(p)$, which moves limb i to gripping point p provided the remaining attached limbs allow the body to be positioned within reach of p, and RELEASE_i, which detaches limb i provided stability is maintained. The goal is to reach a body position from which a designated target location is accessible. This formulation requires no domain-specific search modifications: the LAMA planner (Richter and Westphal 2008) is used directly on the propositional encoding.

Transition Links. Kinematic actions alone cannot bridge large gaps between gripping point clusters, which require dynamic maneuvers such as ballistic jumps. A Transition Link (TL) $\tau = \langle P_{\text{pre}}, P_{\text{post}} \rangle$ is a symbolic action encoding one such maneuver: its precondition specifies the required source configuration, and its effect specifies the landing configuration. Feasibility is verified offline by solving the ballistic equations and checking kinematic and velocity constraints; only validated TLs are injected into the planning instance. The planner treats TLs identically to kinematic actions.

Pruning Strategies. Exhaustive TL generation yields in-

tractably large action sets. We introduce three satisfiability-preserving strategies: *short-TL pruning* removes TLs whose target is already reachable by kinematic actions; *same-component pruning* removes TLs between configurations in the same connected component of the gripping-point reachability graph; and *same-end-center pruning* deduplicates TLs whose targets expose identical sets of reachable gripping points. Together these reduce TL counts by up to 95%.

Physics Simulator and Controller. A 2D rigid-body simulator (PyMunk) models the body, spring-jointed limbs, and contact dynamics. A controller translates planner output into limb-level commands, executing precomputed TL trajectories for dynamic maneuvers and simple attach/release sequences for kinematic steps.

System Properties

Robustness. Correctness is enforced at two layers. Every TL is validated offline: a trajectory exists only if the ballistic equations and kinematic constraints are jointly satisfied, so no infeasible dynamic action can enter the planning instance. At the symbolic level, the FDR formulation guarantees that any plan returned by the planner is executable under the model. The physics simulator then serves as a ground-truth check: the demonstration shows planned sequences executing in PyMunk, making failures immediately visible.

Generalization. The planning model is parameterized entirely by gripping point coordinates, robot reach E, limb count n, and stability threshold k. Any terrain layout expressible in these terms produces a valid planning instance automatically, with no hand-crafted domain knowledge. The demonstration includes multiple distinct terrain configurations to illustrate this directly.

Scalability. The bottleneck is TL precomputation, whose cost grows with gripping point density. Pruning is what makes the approach practical: without it, TL counts exceed 30,000 on larger instances and the planner times out; with same-component pruning alone, counts drop by up to 95% and previously intractable instances become solvable. Instance 10 in our benchmark remains unsolved under all configurations, marking a concrete open challenge.

Deployment Effort. No learning, training data, or hyperparameter tuning is required. The only inputs are the geometric parameters above. Precomputation is a one-time offline step per environment; the planner and controller then run on commodity hardware (experiments used an Intel Core i7-6700K).

Demonstration

The live demonstration will show the full MonkeyBot pipeline: gripping point configuration input, TL precomputation and pruning, plan generation, and physics-based execution in PyMunk. Attendees will be able to modify gripping point layouts and observe how the system adapts its plan. A pre-recorded video is available at:

https://www.youtube.com/watch?v=exAnQH9lMtI

References

Helmert, M. 2006. The Fast Downward Planning System. *JAIR*, 26: 191–246.

Micheli, A.; Bit-Monnot, A.; Röger, G.; Scala, E.; Valentini, A.; Framba, L.; Rovetta, A.; Trapasso, A.; Bonassi, L.; Gerevini, A. E.; Iocchi, L.; Ingrand, F.; Köckemann, U.; Patrizi, F.; Saetti, A.; Serina, I.; and Stock, S. 2025. Unified Planning: Modeling, manipulating and solving AI planning problems in Python. *SoftwareX*, 29: 102012.

Nadan, P.; Backus, S.; and Johnson, A. M. 2024. LORIS: A lightweight free-climbing robot for extreme terrain exploration. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 18480–18486. 3 Park Avenue, New York: IEEE.

Parness, A.; Abcouwer, N.; Fuller, C.; Wiltsie, N.; Nash, J.; and Kennedy, B. 2017. Lemur 3: A limbed climbing robot for extreme terrain mobility in space. In *2017 IEEE international conference on robotics and automation (ICRA)*, 5467–5473. 3 Park Avenue, New York: IEEE.

Richter, S.; and Westphal, M. 2008. The LAMA Planner — Using Landmark Counting in Heuristic Search. IPC 2008 short papers, http://ipc.informatik.uni-freiburg.de/Planners.

Tanaka, Y.; Shirai, Y.; Lin, X.; Schperberg, A.; Kato, H.; Swerdlow, A.; Kumagai, N.; and Hong, D. 2022. Scaler: A tough versatile quadruped free-climber robot. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5632–5639. 3 Park Avenue, New York: IEEE.